Studio Nx

Backend(FastAPI/Python) 04 — unexpected_error_handler()

2026-10-07

Backend(FastAPI/Python) 04 — `unexpected_error_handler()`

항목	내용
파일	`studionx/backend/app/core/errors.py`
작업	P1-03 백엔드 공통 — 오류 응답 형식 (처리기 네 개 중 마지막)
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	03 `http_error_handler()` · 05 `decode_access_token()` (토큰 검사)

1. 하는 일

01 ~ 03 이 맡지 않은 **모든 오류**(버그 · DB 끊김 등 예상 못 한 오류)를 500 응답으로 바꾼다. 응답에는 일반 문장만 넣고, 오류 내용은 넣지 않는다.

```
500  {"error": {"code": "INTERNAL_ERROR", "message": "Something went wrong. Please try again.", "details": null}}
```

2. 언제 쓰나

코드가 예상하지 못한 이유로 멈췄을 때다. 정상이라면 일어나지 않아야 하는 오류다.

상황	원래 오류 (서버 안)	응답
코드 버그 (`None` 의 칸을 읽음 등)	`AttributeError: 'NoneType' object has no attribute 'email'`	500 일반 문장
DB 서버가 꺼짐	`OperationalError: (2003, "Can't connect to MySQL server on 'localhost'")`	500 일반 문장
파일 저장 실패	`PermissionError: [Errno 13] … C:\Users\…\uploads\…`	500 일반 문장

오류 내용은 어디에 남나

응답에는 없지만 **사라지지 않는다.** 이 처리기가 응답을 보낸 뒤 Starlette 가 원래 오류를 다시 던지고, 두 곳이 그것을 기록한다.

순서	일어나는 일
① 라우터에서 오류	`RuntimeError(...)`
② 이 처리기	사용자에게 500 일반 문장 응답
③ Starlette	응답을 보낸 뒤 **원래 오류를 다시 던짐** (`raise exc`)
④ uvicorn	서버를 실행한 터미널에 오류 전체(어느 파일 몇째 줄인지) 출력
⑤ Sentry	`SENTRY_DSN` 이 설정돼 있으면 Sentry 에 보냄 (`core/sentry.py` 가 비밀번호 · 토큰 · 쿠키를 지운 뒤)

누가 보나	보는 것
사용자 (화면)	"Something went wrong. Please try again."
개발자	터미널 로그 · Sentry — 오류 종류 · 내용 · 위치 전체

처리기 네 개 정리 (P1-03 오류 응답 형식 완료)

#	처리기	맡는 오류	상태
01	`app_error_handler`	우리 코드가 일부러 던진 `AppError`	오류에 적힌 값
02	`validation_error_handler`	입력 형식 오류	422
03	`http_error_handler`	없는 주소 · 다른 방식 등	404 · 405 …
04	**`unexpected_error_handler`**	그 밖의 모든 오류	500

3. 코드

import (파일 맨 위)

```
from enum import StrEnum

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from starlette.exceptions import HTTPException as StarletteHTTPException

from app.schemas.common import ErrorBody, ErrorResponse
```

`errors.py` 맨 위의 import 이다. 이 함수는 **`Request`** · **`JSONResponse`** 만 쓴다. `Exception` 은 파이썬 기본이라 import 가 필요 없다.

함수

```
async def unexpected_error_handler(request: Request, exc: Exception) -> JSONResponse:
    return error_json(500, ErrorCode.INTERNAL_ERROR, "Something went wrong. Please try again.")
```

`register_exception_handlers()` 에 등록:

```
app.add_exception_handler(Exception, unexpected_error_handler)
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀 (예: `Request`)

식	뜻
`exc: Exception`	`Exception` 은 거의 모든 오류의 부모. 그래서 01 ~ 03 이 맡지 않은 오류가 모두 여기로 온다
`exc` 를 쓰지 않음	일부러다. 오류 내용을 응답에 넣지 않으므로 받기만 한다
`add_exception_handler(Exception, …)`	FastAPI 는 `Exception` 처리기를 다른 처리기와 따로, **가장 바깥**(`ServerErrorMiddleware`)에 둔다. 더 구체적인 처리기(01 ~ 03)가 먼저 잡고, 아무도 잡지 않은 오류만 여기로 온다

4. 흐름

1. 01 ~ 03 이 잡지 않은 오류가 가장 바깥까지 올라온다.

2. 이 처리기가 500 일반 문장 응답을 만든다.
3. Starlette 가 응답을 보낸 뒤 원래 오류를 다시 던진다 → 터미널 로그 · Sentry 가 기록.

5. 왜 이렇게

선택	이유
오류 내용을 응답에 넣지 않음	오류 문장에는 SQL · 파일 경로 · DB 주소 · 비밀값이 섞일 수 있다 (위 §2 의 예). 공격자에게 서버 구조를 알려 주게 된다 (PRD 01 §5)
기록 코드를 따로 쓰지 않음	Starlette 가 오류를 다시 던져 uvicorn · Sentry 가 이미 기록한다. 여기서 또 기록하면 Sentry 에 같은 오류가 두 번 쌓인다
모양은 다른 오류와 같게	`api.js` 가 `INTERNAL_ERROR` 로 읽는다. 기본 500 응답은 `Internal Server Error` 글자(JSON 아님)라서 프론트가 형식을 알 수 없다
문구는 "다시 시도"	DB 가 잠깐 끊긴 경우처럼 다시 하면 되는 일이 많다. 계속되면 Sentry 에서 알 수 있다

6. 시험 (`tests/test_errors.py` 에 2개 추가)

시험용 주소가 `RuntimeError("SELECT * FROM users WHERE password_hash='…'")` 를 던지게 했다.

경우	결과
응답 확인 (`raise_server_exceptions=False`)	500 · 본문이 정확히 위 JSON · 응답에 `password_hash` · `RuntimeError` 없음
다시 던지는지 확인 (기본 설정)	시험 도구에서 `RuntimeError` 가 그대로 올라옴 → 로그 · Sentry 가 기록할 수 있음

`raise_server_exceptions=False`: 시험 도구(`TestClient`)는 기본으로 서버 오류를 다시 던져 시험을 멈춘다. 실제 응답을 보려고 이 옵션을 끈다.

전체 시험 26개 통과, `ruff` 검사 통과.